Unit Test Automation Summary Report

SQE Assignment 3 | Project: IntellectA | Members: 3062 and 3097 | October 25, 2026

1. Overview

We picked IntellectA, an open-source AI study tutor. Its backend is written in Python with FastAPI and LangGraph, and its frontend is Next.js. We tested only the Python backend, using pytest.

Each of us took three modules. Together we wrote 257 automated tests, and all of them pass in about 9 seconds on Windows with Python 3.12.10.

Repository: https://github.com/muhammad-abdullah-nova-dev/IntellectA

2. Setup (Task 1)

We ran the project on Windows with Python 3.12. The steps were:

1. Clone the repository and open the backend folder.
2. Create a virtual environment with `python -m venv venv` and turn it on.
3. Install the project packages with `pip install -r requirements.txt`.
4. Install the test tools with `pip install pytest pytest-asyncio pytest-xdist pytest-order pytest-cov httpx`.
5. Run `pytest -q` from the backend folder.

The unit tests do not need PostgreSQL, Redis or a Groq key. Calls to the database and to Redis are replaced with small fake objects, so after step 4 the tests run on any machine. The setup video is in `backend/docs/Setup.mp4`.

3. Modules we tested (Task 2)

We chose code with clear rules and clear boundaries, and code that can run without a live service.

Member	Module	Files under test	Tests
3062	Authentication and users	auth/jwt.py, db/repository/users.py	39

Member	Module	Files under test	Tests
3062	Adaptive assessment	assessment/adaptive_engine.py	25
3062	Short-term session memory	memory/short_term.py	33
3062	Data-driven tests (CSV)	tests/test_3062_data_driven.py	18
3097	Settings	config.py	30
3097	Allowed subjects	orchestration/subjects.py	47
3097	Graph routing	orchestration/graph/routing.py	46
3097	Data-driven tests (CSV)	tests/test_3097_data_driven.py	19

Member 3062 wrote 115 tests and member 3097 wrote 142, which makes 257 in total.

4. How we designed the tests

We wrote notes for every module before writing the tests. They are in `backend/notes`. Each note has tables with a test ID, the technique, the input and the expected result. Every test has its ID in its docstring, so it is easy to go from a note to a test and back.

These are the techniques we used, each with a real example from our tests:

- **Equivalence partitioning (ECP):** login tokens were split into valid, expired, signed with another secret, signed with another algorithm, and not a token at all.
- **Boundary value analysis (BVA):** scores 0.39, 0.4, 0.69 and 0.7 around the difficulty thresholds, and response times of 4999, 5000, 30000 and 30001 ms.
- **Error guessing:** a `None` token, a `NaN` score, the number 123 where text is expected, and a CORS value made only of commas.
- **State machine:** a token that goes from valid to expired, a difficulty that climbs from 2.5 to 5.0 in five perfect answers, a settings cache that goes from cached to cleared, and a graph path from the router to an agent to the memory writer.
- **Decision table:** which node comes after the router depending on scope, intent and subject, and whether a subject text is allowed and what label it gets.

For every function the notes list between 5 and 10 cases. Each module also has between 5 and 9 module-level cases. The pytest count is higher than the number of rows in the notes because one parametrized test runs once for each input.

5. Data-driven tests (Task 3)

We have four CSV files in `backend/data`. The test code reads them with `csv.DictReader` and passes the rows to `@pytest.mark.parametrize`, so each row becomes one test with its own ID, such as `3062-CSV-DIFF-04`.

CSV file	Function it feeds	Rows
3062_difficulty_data.csv	compute_next_difficulty	12
3062_bad_tokens_data.csv	decode_access_token	6
3097_routing_data.csv	route_after_router	12
3097_cors_data.csv	cors_origin_list	7

That gives 37 data-driven tests. Each member also has an Excel sheet in `backend/docs` (`3062_test_cases.xlsx` with 18 cases and `3097_test_cases.xlsx` with 19 cases). The sheet lists the input, the expected result, the data file and the status of every CSV case.

6. Running the tests (Task 4)

Groups. Tests are grouped with pytest markers, which are listed in `pytest.ini`. There are 28 smoke tests, 37 data-driven tests, 47 boundary tests and 6 security tests. The other markers are `regression` (everything), `auth`, `assessment`, `memory`, `config`, `subjects`, `routing` and `slow`.

Commands.

```
pytest                          all tests
pytest -m smoke                 one group
pytest -m "auth or memory"      two groups together
pytest -n 4 --dist loadfile     parallel run
python run_tests.py smoke       shortcut for a group
```

Order. We set the order with pytest-order. Smoke tests run first (order 1). Then the first, second and third module of each member (orders 2, 3 and 4). Then the CSV tests (order 5). The one slow test, which waits 3 seconds for a token to expire, runs last (order 6). The order shows in the output of a normal run.

Parallel. We use pytest-xdist with 4 workers. The option `--dist loadfile` keeps all tests of one file on the same worker. Our suite is small, so the parallel run is not faster (about 10 seconds against 9). In a parallel run the global order is not guaranteed.

Parameterization. Most test files use `@pytest.mark.parametrize`, for example for password lengths, score boundaries, limit values, subject names and the CSV rows.

Each member has a how-to file with the same information: `backend/docs/3062_HOW_TO_RUN_TESTS.md` and `backend/docs/3097_HOW_TO_RUN_TESTS.md`.

7. Results

We ran the full suite on Windows with Python 3.12.10 and pytest 9.1.1.

	Tests	Passed	Failed
Member 3062	115	115	0
Member 3097	142	142	0
Total	257	257	0

The whole run takes about 9 seconds. Three of those seconds come from the single slow test.

8. What we noticed about the code

While writing the tests we found some behavior that looks odd. We did not change the project code. Our tests record what the code does today.

- `get_messages` with `limit=0` asks Redis for the whole list, not for zero messages.
- A wrong answer with a high score (0.7 or more) is treated as a partial answer and lowers the difficulty by 0.1.
- `get_difficulty` returns a stored value such as 0.0 without keeping it inside the 1.0 to 5.0 range.
- `create_user_with_password` has no check on password length, so any rule must live in the API layer.
- `uses_upstash_rest` is true when the url and token contain only spaces.

- `subject_label` does not trim spaces, so `" physics "` comes back as `" Physics "`.
- `get_settings` keeps its old value after an environment change until the cache is cleared or `reload_settings` is called.

9. Limits

- The database and Redis are replaced with fakes, so we did not test real connections. The Groq calls and the embedding model are not tested either.
- We tested only the Python backend. The Next.js frontend is not covered.
- We covered six modules. Others, such as the agents, the API routes, retrieval and long-term memory, have no tests yet.
- The example tests in the repository's `UNIT_TESTING_GUIDE.md` did not match the current function signatures, so we wrote every test from the source code instead.

10. Repository layout

```
backend/
  src/intellecta/      project code
  tests/               conftest.py, test_3062_*.py, test_3097_*.py
  data/                the four CSV files
  docs/                setup video, Excel sheets, how-to files
  notes/               test design notes, three per member
  pytest.ini           markers and test settings
  run_tests.py         shortcuts for running groups
```

11. Who did what

Member	Work
3062	Authentication and users, adaptive assessment and short-term memory: notes, 115 tests, two CSV files, Excel sheet, how-to file, setup video
3097	Settings, allowed subjects and graph routing: notes, 142 tests, two CSV files, Excel sheet, how-to file

Member	Work
Both	The shared files `pytest.ini`, `run_tests.py` and `tests/conftest.py`

The Git history of the repository shows the commits of each member.